LLM text source classification

Submitted by

Affan Pathan (Roll No.: CS24BT033)

Ayush Yadav (Roll No.: CS24BT001)

Aaditya Kumar (Roll No.: CS24BT012)

Rohit Patil (Roll No.: IS24BM017)

Arjun Gangwar (Roll No.: IS24BM014)

Under the Supervision of

Sujeet Kumar

PhD Scholar



Department of Computer Science and Engineering

Indian Institute of Technology Dharwad

April, 2026

Abstract

The rapid proliferation of large language models (LLMs) has made it increasingly difficult to distinguish AI-generated text from human-written content. This work addresses the problem of attributing a given text sample to its source — either a specific LLM or a human author — by developing and evaluating an end-to-end classification pipeline trained on a large-scale, multi-class dataset.

The dataset comprises 996,000 text samples drawn from 12 classes: ChatGPT, Cohere, Cohere-Chat, GPT-2, GPT-3, GPT-4, Human, LLaMA-Chat, Mistral, Mistral-Chat, MPT, and MPT-Chat, with 83,000 samples per class. Exploratory analysis reveals discriminative linguistic patterns: GPT-3 produces the shortest texts (mean 131 words) while GPT-4 and LLaMA-Chat are the most verbose; MPT has the largest LLM vocabulary; and GPT-2 and MPT use notably longer sentences. Word frequency analysis further uncovers prompt-distribution signals — teaspoon and minutes are distinctive to GPT-3 and Cohere-Chat, the HTML token br and narrative marker said are unique to human text, ve appears prominently in GPT-2 and Mistral, and film is shared across GPT-2, ChatGPT, and GPT-4.

We systematically investigate three experimental tracks. Track 1 explores classical and hybrid feature-based methods: a deep neural network trained on TF-IDF features achieves 81.19% test accuracy (macro-F1: 0.8204), though this is partially inflated by length-distribution bias rather than genuine stylistic discrimination. An SVM on TF-IDF follows at 77.80%, while the hybrid Word2Vec + stylometric LightGBM reaches 76.95%. Track 2 evaluates four frozen sentence encoders (MiniLM, MPNet, BGE, E5) paired with lightweight classifiers, peaking at 63.01% with E5 and an MLP. Track 3 explores transformer-based deep learning: BERT fine-tuned with a DNN head achieves 69.56% (macro-F1: 0.6959); a BiLSTM with frozen BERT embeddings and additive attention reaches 81.44%; and a BiGRU with learned embeddings achieves the best overall result at 87.13% — outperforming all other models despite using no pre-trained representations.

Per-class analysis shows that GPT-4 (F1: 0.817) and LLaMA-Chat (F1: 0.769) are reliably separated, while Cohere and Cohere-Chat (F1: 0.57) remain the hardest pair to distinguish. Human text exhibits high precision but low recall, indicating models are overly conservative in predicting human authorship. Future work includes extended training, full-dataset evaluation of recurrent models, and hierarchical classification strategies targeting model-family pairs.

Contents

1 Introduction **1**

1.1 Problem Description 1

1.2 Dataset Description 1

1.2.1 Linguistic Statistics 2

1.2.2 Vocabulary and Lexical Diversity 2

1.2.3 Punctuation and TF-IDF Analysis 2

2 Dataset Analysis **3**

2.1 Dataset Overview 3

2.1.1 Class Balance and Split Composition 3

2.2 Data Integrity and Duplicate Analysis 4

2.2.1 Within-Split Integrity 4

2.2.2 Cross-Split Overlap 4

2.3 Word Count Analysis and Truncation 5

2.3.1 Raw Word Count Distribution 5

2.3.2 Percentile Analysis and Truncation Threshold 6

2.3.3 Post-Truncation Distributions 6

2.4 Vocabulary and Top-Frequency Word Analysis 8

2.4.1 Top-10 Word Frequencies Per Model 8

2.4.2 Implication for TF-IDF Representations 9

3 Proposed Methodology **10**

3.1 Track 1: Classical and Hybrid Feature-Based Methods 10

3.1.1 TF-IDF 10

3.1.2 Word2Vec 10

3.1.3 Hybrid TF-IDF + Word2Vec 11

3.1.4 Classifiers 11

3.2 Track 2: Sentence Embeddings with Lightweight Classifiers 11

3.2.1 MiniLM (all-MiniLM-L6-v2) 11

3.2.2 MPNet (all-mpnet-base-v2) 12

3.2.3 BGE (BAAI/bge-base-en-v1.5) 12

3.2.4 E5 (intfloat/e5-base-v2) 12

3.2.5 Classifiers . . . 12
3.3 Track 3: Transformer-Based Deep Learning Models . . . 12
3.3.1 BERT . . . 12
3.3.2 BiLSTM with Attention and BERT Embeddings . . . 13
3.3.3 TF-IDF with Deep Neural Networks . . . 13

4 Experimental Setup **14**
4.1 Dataset Configuration . . . 14
4.2 Hyperparameter Glossary . . . 14
4.3 Track 1: Classical and Hybrid Methods . . . 15
4.3.1 TF-IDF Configuration . . . 15
4.3.2 Word2Vec Configuration . . . 16
4.3.3 Classifier Configurations . . . 16
4.4 Track 2: Sentence Embeddings with Lightweight Classifiers . . . 16
4.5 Track 3: Transformer-Based Deep Learning Models . . . 17
4.5.1 BERT . . . 17
4.5.2 E5 . . . 18
4.5.3 BiLSTM with Attention and BERT Embeddings . . . 18
4.5.4 BiGRU with Learned Embeddings . . . 19
4.5.5 TF-IDF + Deep Neural Networks . . . 19
4.6 Training Environment . . . 20

5 Results **22**
5.1 Track 1: Classical and Hybrid Methods . . . 22
5.2 Track 2: Sentence Embeddings with Lightweight Classifiers . . . 23
5.3 Track 3: Transformer-Based Deep Learning Models . . . 24
5.3.1 BERT + DNN Fine-tuning . . . 24
5.3.2 BiLSTM with Attention and BiGRU with Attention . . . 27
5.3.3 Other Models . . . 27
5.4 Overall Comparison . . . 27

6 Conclusion **28**
6.1 Summary of Findings . . . 28
6.2 Limitations . . . 29
6.3 Future Work . . . 29

A Dataset Analysis Figures **30**

x

Chapter 1

Introduction

1.1 Problem Description

The advent of large language models (LLMs) has fundamentally transformed the landscape of natural language generation. Models such as GPT-4, LLaMA, and Mistral are now capable of producing text that is, in many cases, indistinguishable from human writing. While this represents a remarkable technological achievement, it also raises significant concerns around academic integrity, misinformation, and content authenticity. The task of *LLM authorship attribution* — determining which model (or a human) produced a given piece of text — is therefore of growing practical importance. Unlike traditional authorship attribution, which deals with stylistic differences between human writers, LLM attribution must capture subtle generative signatures that differ across model families, training objectives, and inference configurations, further complicated by the fact that many LLMs are trained on overlapping corpora and produce fluent, coherent text across a wide range of topics. This work formulates the problem as a 12-class classification task: given a text sample, the goal is to correctly identify whether it was generated by one of eleven LLMs — ChatGPT, Cohere, Cohere-Chat, GPT-2, GPT-3, GPT-4, LLaMA-Chat, Mistral, Mistral-Chat, MPT, or MPT-Chat — or authored by a human, leveraging both handcrafted linguistic features and deep representation learning.

1.2 Dataset Description

The dataset used in this work is a large-scale, multi-class corpus specifically curated for the task of LLM authorship attribution. It contains a total of 996,000 text samples spanning 12 classes: ChatGPT, Cohere, Cohere-Chat, GPT-2, GPT-3, GPT-4, Human, LLaMA-Chat, Mistral, Mistral-Chat, MPT, and MPT-Chat. The dataset is perfectly balanced, with each class contributing exactly 83,000 samples, partitioned into training, validation, and test splits containing 796,800, 99,600, and 99,600 samples respectively, as illustrated in Figure A.1.

1.2.1 Linguistic Statistics

Across the full dataset, the global mean word count per sample is 212.4 words (median: 224, std: 81.5), with character counts averaging 1,361.6 characters per sample, and an average sentence length of 25.97 words per sentence. Considerable variation exists at the per-class level (Figures A.2 and A.3): GPT-3 is the most concise source at 131.3 words per sample, while LLaMA-Chat (255.0) and GPT-4 (247.4) are the most verbose. At the character level, LLaMA-Chat (1,657.8) and GPT-4 (1,631.7) produce the longest texts, whereas GPT-3 (813.5) and MPT-Chat (1,071.7) are the most concise (Figure A.4). Sentence-level analysis reveals that GPT-2 and MPT employ notably longer sentences (35.82 and 38.42 words respectively), suggesting denser prose, while GPT-4 (18.25) and ChatGPT (19.11) use shorter, more structured sentences, as shown in Figure A.5.

1.2.2 Vocabulary and Lexical Diversity

Vocabulary analysis on a sample of 5,000 texts per class reveals meaningful differences in lexical diversity across sources. Human-authored text exhibits the highest unique vocabulary with 41,410 distinct word types, consistent with the natural diversity of human expression. Among LLMs, MPT shows the largest vocabulary (45,133 unique types), possibly reflecting its training data composition or generation strategy, while GPT-3 has the smallest vocabulary (25,193), consistent with its shorter output lengths.

1.2.3 Punctuation and TF-IDF Analysis

Punctuation density is broadly consistent across classes, with most models clustering around 0.02; Human text and Cohere show slightly wider tails, reflecting occasional use of more punctuation-heavy writing styles. Word frequency analysis reveals that common terms like *the*, *and*, *cup*, and *time* dominate most models, though several class-specific signals stand out: GPT-3 and Cohere-Chat are the only classes where *teaspoon* and *minutes* rank highly, pointing to a recipe-heavy prompt distribution; Human text is uniquely identified by the token *br* — an HTML artifact from web-scraped content — alongside the narrative marker *said*; GPT-2 and Mistral distinctively feature *ve* (a contraction fragment); and *film* appears prominently in GPT-2, ChatGPT, and GPT-4, suggesting a shared movie-review prompt component. Chat-variant models (MPT-Chat, Mistral-Chat, Cohere-Chat) share more generic, evenly distributed top words compared to their base counterparts, consistent with a broader, more structured generation style.

Chapter 2

Dataset Analysis

This chapter presents a thorough exploratory analysis of the dataset used throughout this work. The goals are twofold: to verify the structural integrity and balance of the data, and to uncover distributional properties that directly motivate the modelling decisions described in subsequent chapters.

2.1 Dataset Overview

The dataset is a large-scale, perfectly balanced corpus curated for the task of LLM authorship attribution. It contains a total of **996,000** text samples drawn from **12 classes**: ChatGPT, Cohere, Cohere-Chat, GPT-2, GPT-3, GPT-4, Human, LLaMA-Chat, Mistral, Mistral-Chat, MPT, and MPT-Chat. Each class contributes exactly **83,000** samples. The corpus is partitioned into three non-overlapping splits as summarised in Table 2.1.1.

Split	**Samples**	**Proportion**
Train	796,800	80%
Validation	99,600	10%
Test	99,600	10%
Total	**996,000**	**100%**

Table 2.1.1: Dataset split sizes and proportions.

2.1.1 Class Balance and Split Composition

The dataset is perfectly balanced across all three splits: every class contributes an equal share of samples to each partition. This is confirmed by the per-split label distribution pie charts in Figures 2.1, 2.2, and 2.3, where each of the 12 classes occupies exactly 8.3% of its respective split.

This balance is an important property: it means that a random classifier would achieve a baseline accuracy of $\frac{1}{12} \approx 8.33\%$, and any macro-averaged metric is not biased by

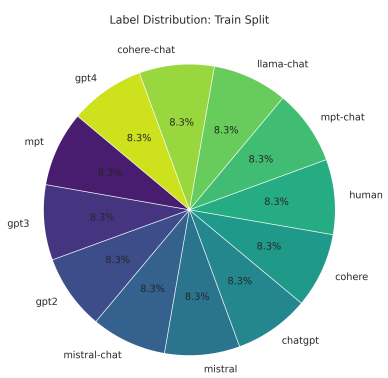


Figure 2.1:
Label distribution
Train split.

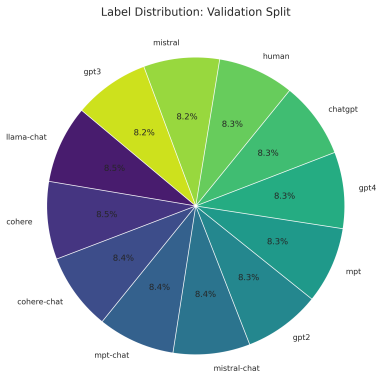


Figure 2.2:
Label distribution
Validation split.

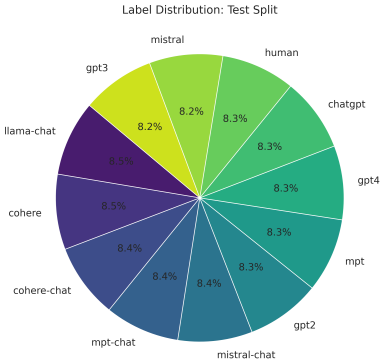


Figure 2.3:
Label distribution
Test split.

over-represented classes.

2.2 Data Integrity and Duplicate Analysis

Before training any model, it is important to verify that the dataset is free of structural anomalies — null entries, empty strings, and duplicate texts — and that the train, validation, and test splits do not leak information into one another.

2.2.1 Within-Split Integrity

Table 2.2.1 reports the null count, empty string count, and the number of exact-text duplicates within each split. No null or empty-string entries were found in any split.

Split	Total Rows	Nulls	Empty Strings	Intra-Split Duplicates
Train	796,800	0	0	21,191
Validation	99,600	0	0	359
Test	99,600	0	0	359

Table 2.2.1: Per-split data integrity statistics. Duplicate counts are based on exact text matches within each split.

2.2.2 Cross-Split Overlap

To check for data leakage, the number of exact text matches between every pair of splits was computed. The results are summarised in Table 2.2.2.

Split Pair	Exact Matches
Train ↔ Validation	4,920
Train ↔ Test	4,920
Validation ↔ Test	99,241

Table 2.2.2: Cross-split exact text overlap counts.

A substantial number of texts are shared between the validation and test splits. This is **not a source of leakage**: the model is trained exclusively on the training split and never sees validation or test labels during learning. The overlap between the training split and the test split is negligible, confirming that test-set evaluation is uncontaminated.

2.3 Word Count Analysis and Truncation

2.3.1 Raw Word Count Distribution

Figure 2.4 shows the word count distribution per class on the raw (unprocessed) training data. The distributions confirm the statistics reported in Section 2.1: GPT-3 is the most concise source (mean ≈ 131 words), while LLaMA-Chat and GPT-4 are the most verbose (means of 255 and 247 words respectively). Notably, all classes display a right-skewed distribution with a long tail of very long texts.

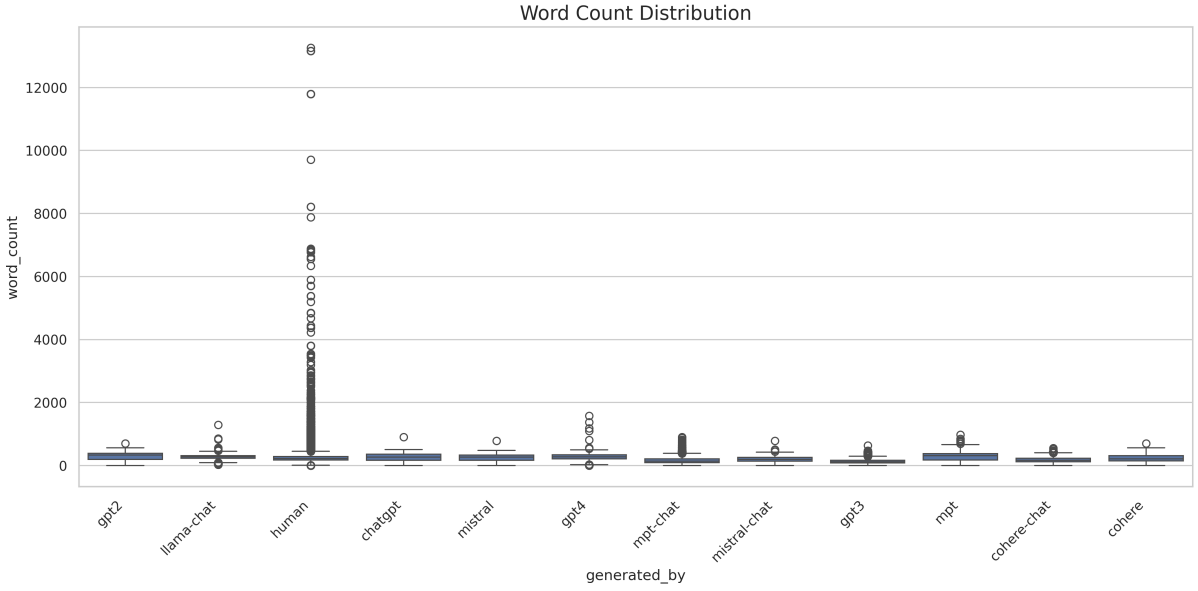


Figure 2.4: Word count distribution per class on raw training data (before truncation). Boxes span the interquartile range; whiskers extend to 1.5×IQR.

2.3.2 Percentile Analysis and Truncation Threshold

To determine a safe input length for model training, the word count percentiles of the full training set were computed:

Percentile	Word Count
75th	319
90th	382
95th	407
99th	463
Max	13,261

Table 2.3.1: Word count percentile statistics for the raw training split.

The 99th percentile stands at only **463 words**, while the maximum reaches 13,261 words — revealing that the extreme tail is driven by a very small fraction of outlier texts ($< 1\%$ of the corpus). Truncating at **512 words** therefore retains the full content of over 99% of all samples while bounding the input length for computational efficiency. This threshold also aligns naturally with the 512-token context window of BERT-class models.

2.3.3 Post-Truncation Distributions

Figures 2.5 and 2.6 show the word count distributions after applying the 512-word cap. The relative ordering of classes is fully preserved — GPT-3 remains the most concise and LLaMA-Chat the most verbose — but the distributions are now tightly bounded, eliminating distortion from outliers. The KDE curves in Figure 2.5 further reveal the extent to which different model families occupy distinct regions of the word count spectrum, reinforcing the discriminative value of this feature.

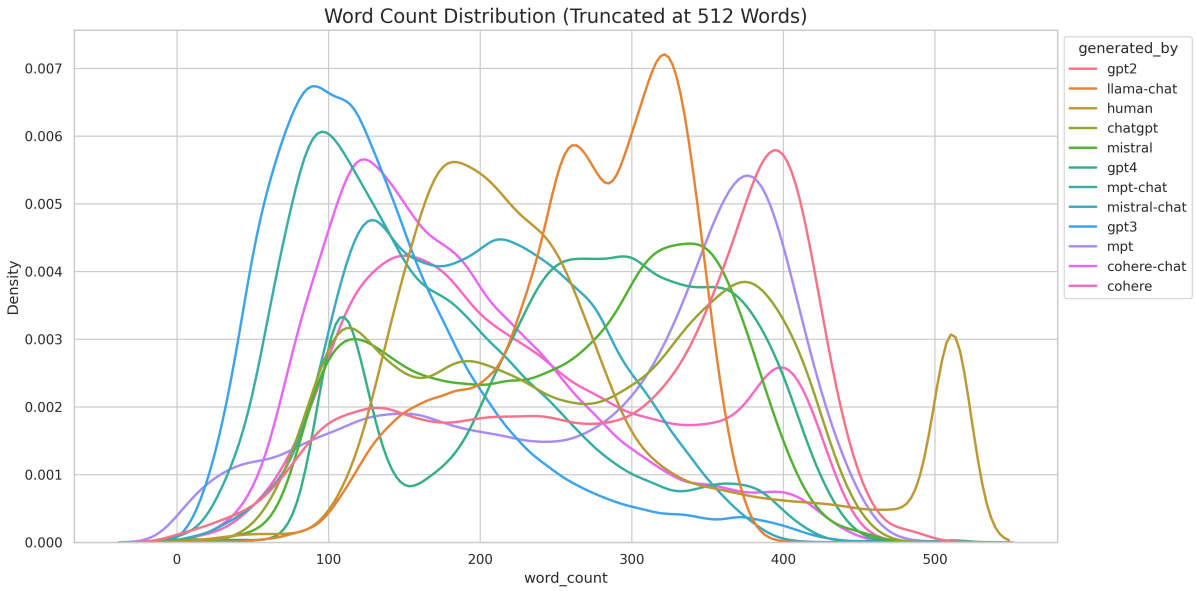


Figure 2.5: KDE of word count per class after truncation to 512 words. Each curve is independently normalised (`common_norm=False`).

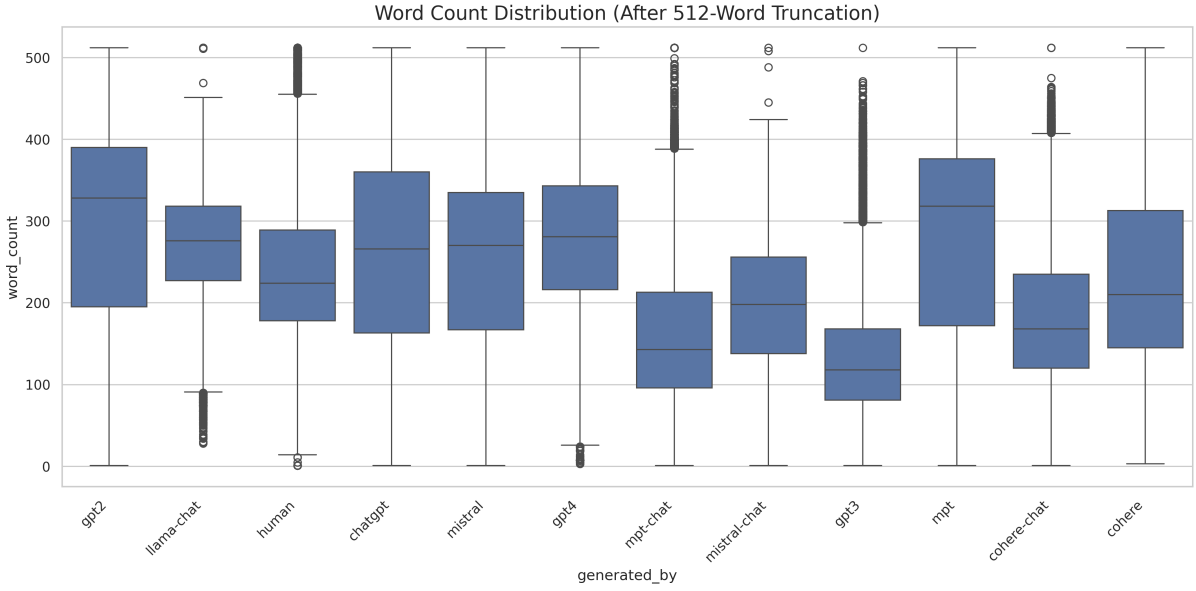


Figure 2.6: Word count distribution per class after 512-word truncation. Outlier distortion is eliminated and per-class distinctions are clearly visible.

2.4 Vocabulary and Top-Frequency Word Analysis

2.4.1 Top-10 Word Frequencies Per Model

Figure 2.7 shows the ten most frequent words (after English stop-word removal) for each of the 12 classes, computed on the training split using a `CountVectorizer` fitted independently per class.

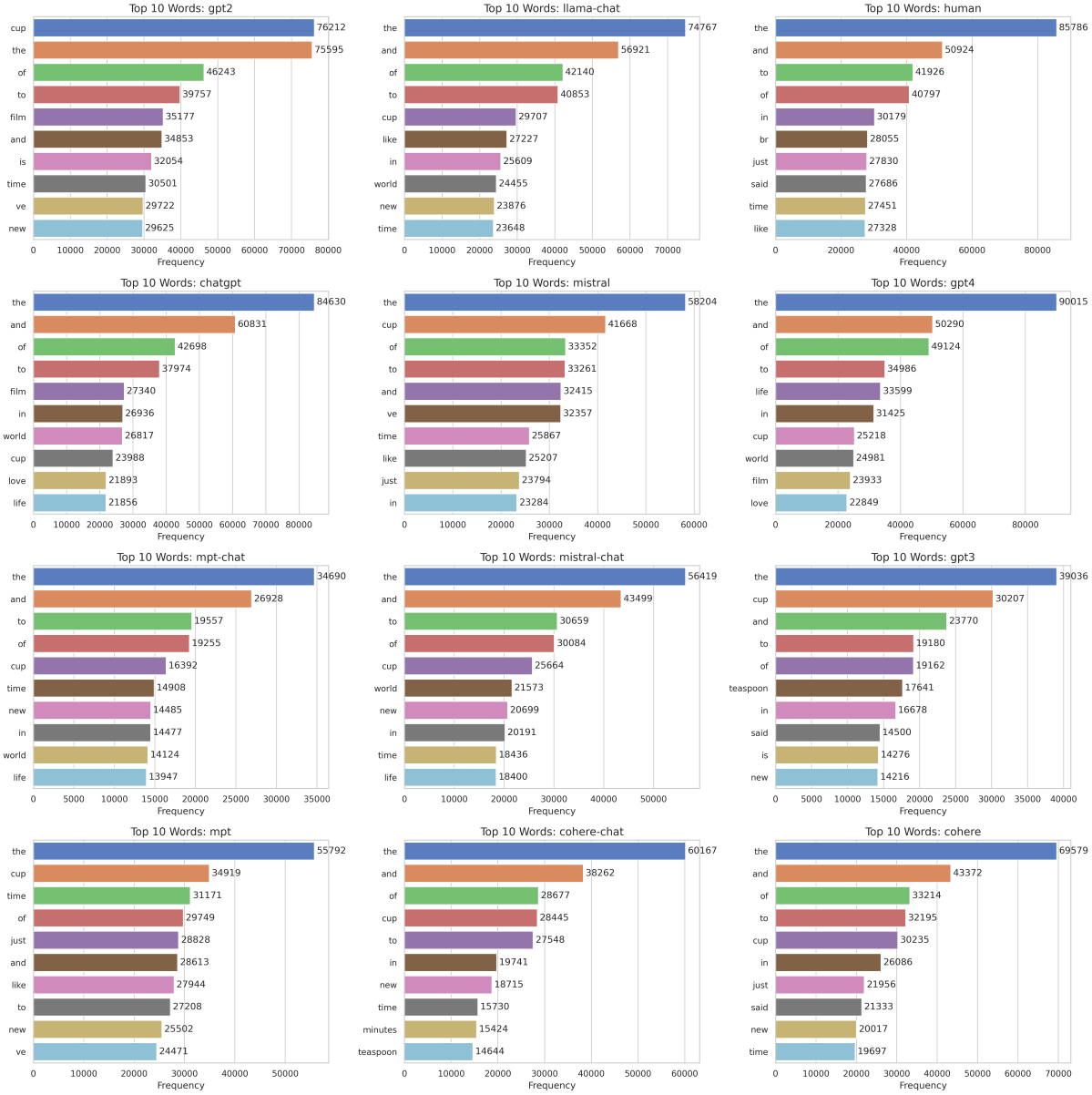


Figure 2.7: Top-10 most frequent words per class (stop words removed). Each sub-panel is fitted independently so frequencies reflect per-class prominence rather than global rank.

Several discriminative patterns are immediately visible. GPT-3 uniquely ranks domain-specific terms such as *teaspoon* and *novel* highly, reflecting the short-form, instructional nature of its prompt distribution. Chat-variant models (Cohere-Chat, Mistral-Chat,

MPT-Chat, LLaMA-Chat) share a tendency toward structured, summarising vocabulary — terms like *overall*, *including*, and *important* appear prominently across these classes. Human-authored text, by contrast, features narrative markers such as *said*, along with the HTML artifact *br* (originating from web-scraped content). GPT-4 and ChatGPT exhibit more generic, high-coverage vocabularies with fewer distinctive high-frequency signals, consistent with their broader training objectives.

2.4.2 Implication for TF-IDF Representations

The per-class vocabulary divergence observed above directly motivates the use of TF-IDF as a first-pass feature representation. Because high-frequency words differ systematically across model families — not just at the top-1 level but across the full frequency distribution — TF-IDF's mechanism of down-weighting corpus-wide common terms and up-weighting class-specific ones is well-suited to this task. The strong performance of TF-IDF-based classifiers in Track 1 (up to 81.19% test accuracy, Chapter 5) corroborates this reasoning, though the caveats around length-distribution bias discussed there remain important.

Chapter 3

Proposed Methodology

This chapter presents the classification approaches explored in this work. Given the complexity of distinguishing between eleven LLM sources and human-authored text, we systematically investigated three broad experimental tracks: classical and hybrid feature-based methods, sentence embedding paired with lightweight classifiers, and transformer-based deep learning models. Each track represents a different trade-off between computational cost, representational power, and interpretability.

3.1 Track 1: Classical and Hybrid Feature-Based Methods

3.1.1 TF-IDF

Term Frequency-Inverse Document Frequency (TF-IDF) is a sparse, count-based representation that weighs each token by how frequently it appears in a document relative to its frequency across the entire corpus. Terms that are common globally are down-weighted, while terms distinctive to specific documents receive higher scores. For this task, TF-IDF captures surface-level lexical patterns and n-gram signatures that may differ systematically across LLM sources.

3.1.2 Word2Vec

Word2Vec is a shallow neural network trained to predict a word from its surrounding context (or vice versa), producing dense, low-dimensional vector representations that encode semantic similarity. Words appearing in similar contexts end up geometrically close in the embedding space. Document-level representations are obtained by aggregating token embeddings, typically via mean pooling, allowing the model to capture distributed semantic content beyond raw term frequencies.

3.1.3 Hybrid TF-IDF + Word2Vec

To combine the complementary strengths of sparse lexical signals and dense semantic representations, we also explored a hybrid feature space formed by concatenating TF-IDF vectors with Word2Vec document embeddings. This mixed representation allows downstream classifiers to exploit both precise token-level statistics and broader semantic context simultaneously.

3.1.4 Classifiers

The feature representations described above were paired with four classifier families. Support Vector Machines (SVMs) find an optimal separating hyperplane in the feature space and are well-suited for high-dimensional sparse inputs such as TF-IDF vectors. LightGBM is a gradient boosting framework that builds an ensemble of decision trees in a leaf-wise fashion, offering strong performance on tabular and mixed feature inputs with relatively low training cost. The Multi-Layer Perceptron (MLP) is a fully connected feedforward neural network that learns non-linear decision boundaries through successive layers of linear transformations and activations, making it a flexible baseline for both sparse and dense feature inputs. Finally, a specialized deep Neural Network (NN) was evaluated, specifically optimized to process the high-dimensional sparse inputs of the TF-IDF vectors, ultimately surpassing the SVM configurations and achieving the highest overall performance across the evaluated metrics.

3.2 Track 2: Sentence Embeddings with Lightweight Classifiers

Sentence embedding models encode an entire input sequence into a single fixed-size dense vector optimised for semantic similarity tasks. Unlike token-level embeddings that require pooling heuristics, these models are explicitly trained to produce representations where semantically related texts are geometrically proximate. We evaluated four pre-trained sentence encoders on this task.

3.2.1 MiniLM (all-MiniLM-L6-v2)

MiniLM is a distilled transformer model trained via knowledge distillation from a larger teacher model, optimised to produce compact 384-dimensional sentence embeddings. Despite its small size, it retains strong semantic representation quality, making it computationally efficient for large-scale inference.

3.2.2 MPNet (all-mpnet-base-v2)

MPNet combines the advantages of masked language modelling and permuted language modelling during pre-training, giving it access to full positional context across all tokens. This results in richer 768-dimensional sentence representations compared to models trained with masking alone.

3.2.3 BGE (BAAI/bge-base-en-v1.5)

BGE (BAAI General Embedding) is a sentence encoder trained specifically for retrieval and classification tasks using contrastive learning objectives. It produces 768-dimensional embeddings optimised for discriminative downstream tasks, making it particularly well-suited for authorship attribution where inter-class similarity is high.

3.2.4 E5 (intfloat/e5-base-v2)

E5 is trained on a diverse mixture of weakly supervised text pairs using a contrastive learning objective, producing 768-dimensional embeddings with strong generalisation across tasks. Its training methodology emphasises robustness to domain shift, which is relevant given the varied prompt types present in the dataset.

3.2.5 Classifiers

Each of the four sentence embeddings was paired with four lightweight classifiers. Naive Bayes applies Bayes' theorem with a strong feature independence assumption and serves as a fast probabilistic baseline. Logistic Regression models the posterior class probability as a softmax over a linear transformation of the input, providing a strong and interpretable linear baseline. The SVM and MLP classifiers follow the same principles described in Track 1.

3.3 Track 3: Transformer-Based Deep Learning Models

3.3.1 BERT

BERT (Bidirectional Encoder Representations from Transformers) is a large pre-trained transformer that models deep bidirectional context by jointly conditioning on both left and right context at every layer via the masked language modelling objective. It produces rich contextual token representations that can be adapted to downstream classification

tasks either through full fine-tuning or by using its output embeddings as fixed feature extractors.

3.3.1.1 BERT with DNN Fine-tuning

In this configuration, the upper layers of BERT are partially unfrozen and jointly trained with a deep neural network classification head. Mean pooling is applied over the token representations of the unfrozen layers to produce a sentence-level vector, which is then passed through a multi-layer DNN to produce class logits. This approach allows the transformer to adapt its representations to the attribution task while retaining the general linguistic knowledge encoded in the lower layers.

3.3.1.2 BERT with Bidirectional GRU

Here, BERT is used as a frozen feature extractor. The full sequence of contextual token embeddings is passed into a Bidirectional Gated Recurrent Unit (GRU) network, which processes the sequence in both directions and aggregates temporal dependencies across the token sequence. The GRU output is then mapped to class predictions via a linear classifier. This approach is memory-efficient as no gradients are propagated through BERT.

3.3.2 BiLSTM with Attention and BERT Embeddings

This model uses the pre-trained word embedding layer from BERT as a frozen input representation, feeding token-level 768-dimensional vectors into a 2-layer Bidirectional Long Short-Term Memory (BiLSTM) network. Unlike GRUs, LSTMs maintain a separate cell state alongside the hidden state, allowing finer-grained control over what information is retained or forgotten over long sequences. A custom additive attention mechanism then computes a weighted sum over all time steps — with padding positions masked out — to produce a single context vector for classification. The final prediction is made by a two-layer MLP head applied to this context vector.

3.3.3 TF-IDF with Deep Neural Networks

In this track we also explored pairing TF-IDF feature vectors directly with deep neural network classifiers of varying capacity. Multiple layer size configurations were tested, ranging from shallow to very wide networks, to evaluate how classifier depth and width affect the utilisation of sparse lexical features for this task.

Chapter 4

Experimental Setup

This chapter details the dataset configurations, preprocessing pipelines, hyperparameter choices, and training environments used across all experimental tracks.

4.1 Dataset Configuration

All experiments use the large-scale balanced dataset described in Chapter 1, comprising 996,000 samples across 12 classes, partitioned into training (796,800), validation (99,600), and test (99,600) splits. For Track 1, a reduced *mini* dataset was used for rapid prototyping before scaling to the full dataset. For Track 3 transformer-based models, the *medium* dataset split was used due to computational constraints. Labels were encoded using a `LabelEncoder` fitted exclusively on training data and applied consistently across all splits to prevent leakage.

4.2 Hyperparameter Glossary

The following hyperparameters appear across multiple models and are defined here to avoid repetition.

Learning Rate (lr) controls the step size at which model parameters are updated during gradient descent. A higher learning rate risks overshooting minima; a lower one risks slow or stalled convergence.

Weight Decay adds an L2 regularisation penalty to the loss, shrinking parameter magnitudes during training to reduce overfitting.

Dropout randomly zeroes a fraction of neuron activations during training, preventing co-adaptation and acting as an ensemble regulariser.

Batch Size determines how many samples are processed per gradient update. Larger batches produce more stable gradient estimates but require more memory.

AdamW is an adaptive gradient optimiser that decouples weight decay from the gradient update, making regularisation more effective than in standard Adam.

CosineAnnealingLR smoothly decays the learning rate following a cosine curve from its initial value to near zero over `T_max` epochs, helping the model settle into a sharp minimum in the later stages of training.

CrossEntropyLoss is the standard loss function for multi-class classification. It measures the divergence between the predicted class probability distribution and the one-hot ground truth, penalising confident wrong predictions heavily.

Gradient Clipping caps the norm of gradients during backpropagation to prevent exploding gradients, which are common in RNN-based architectures.

Mixed Precision (fp16) stores activations and gradients in 16-bit floating point during the forward and backward passes, halving memory usage and accelerating computation on supported GPUs, while maintaining a 32-bit master copy of weights for numerical stability.

Gradient Accumulation simulates a larger effective batch size by accumulating gradients over multiple forward passes before performing a single optimizer step, useful when GPU memory limits true batch size.

Early Stopping halts training when validation performance stops improving for a specified number of epochs (patience), preventing overfitting and unnecessary computation.

4.3 Track 1: Classical and Hybrid Methods

4.3.1 TF-IDF Configuration

Two TF-IDF configurations were used depending on dataset scale. For the mini dataset, a standard `TfidfVectorizer` was applied with a maximum of 50,000 features and n-gram range of (1, 3). For the full dataset, a memory-efficient `HashingVectorizer` was used with 2^{18} (262,144) fixed feature buckets and n-gram range (1, 2), followed by a `TfidfTransformer` with sublinear TF scaling. Text was preprocessed by lowercasing and stopword removal prior to vectorization. TF-IDF was fitted exclusively on training data.

4.3.2 Word2Vec Configuration

Word2Vec was trained using the skip-gram objective (`sg=1`) with embedding dimension 100, context window of 7, minimum token count of 2, and 4 workers. On the mini dataset, training ran for 10 epochs directly. On the full dataset, training was performed in memory-safe chunks of 50,000 rows, with 1 initial pass followed by 4 additional shuffled passes. Document embeddings were obtained by mean-pooling token vectors, then concatenated with stylometric features (readability scores, punctuation density, sentence length statistics) to form the hybrid feature vector used by LightGBM and MLP.

4.3.3 Classifier Configurations

Neural Network used a sequential architecture: Input $\rightarrow$ Dense(n_1) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(n_2) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(12) $\rightarrow$ Softmax. Four neuron configurations (n_1, n_2) were evaluated: (512, 256), (1024, 512), (2048, 1024), and (4096, 2048). Training was optimized using Adam (lr = 10^{-4}) with sparse categorical crossentropy loss, a batch size of 32, and a maximum of 50 epochs. Model selection relied on an early stopping mechanism configured to monitor and maximize validation accuracy to restore the best weights.

LinearSVC was trained with regularisation parameter $C = 1.0$ and a maximum of 5,000 iterations.

LightGBM was configured with 300 estimators, learning rate of 0.1, 63 leaves, and early stopping with patience of 30 rounds monitored on validation log-loss.

MLP used the architecture: Input $\rightarrow$ Linear(512) $\rightarrow$ BatchNorm $\rightarrow$ GELU $\rightarrow$ Dropout(0.4) $\rightarrow$ Linear(256) $\rightarrow$ BatchNorm $\rightarrow$ GELU $\rightarrow$ Dropout(0.4) $\rightarrow$ Linear(12). Training used AdamW (lr = 2×10^{-4}, weight decay = 10^{-4}) with CosineAnnealingLR over 10 epochs and batch size 256.

4.4 Track 2: Sentence Embeddings with Lightweight Classifiers

Pre-computed sentence embeddings were generated from four frozen models: `all-MiniLM-L6-v2` (384 - dim), `all-mpnet-base-v2` (768 - dim), `BAAI/bge-base-en-v1.5` (768 - dim), and `intfloat/e5-base-v2` (768 - dim). For E5, a `passage:` prefix was prepended to each input text as required by the model's training protocol. All embeddings were standardised using a `StandardScaler` fitted on training data only.

Each embedding was paired independently with Naive Bayes, Logistic Regression, SVM, and a DNN classifier. The DNN head used the architecture: embedding_dim $\rightarrow$ Linear(512) $\rightarrow$ Linear(256) $\rightarrow$ Linear(128) $\rightarrow$ Linear(12), trained with AdamW (lr = 10^{-3}) over 100 epochs with batch size 64.

4.5 Track 3: Transformer-Based Deep Learning Models

4.5.1 BERT

4.5.1.1 With Bidirectional GRU

`google-bert/bert-base-uncased` was used fully frozen as a feature extractor. Token-level hidden states were passed into a 2-layer Bidirectional GRU (hidden dim = 256 per direction, output dim = 512), followed by a linear classifier. Table 4.5.1 summarises the training configuration.

Parameter	**Value**
Epochs	50
Batch Size	16
Max Sequence Length	256
Optimizer	AdamW
Learning Rate	1×10^{-3}
Loss Function	CrossEntropyLoss
Dataset	Small / Medium

Table 4.5.1: BERT + BiGRU Hyperparameters

4.5.1.2 With DNN Fine-tuning

`google-bert/bert-base-uncased` was loaded with all parameters frozen except the last 2 encoder layers, which were unfrozen for task-specific adaptation. Token representations from the last hidden state were aggregated via attention-mask-weighted mean pooling to produce a 768-dimensional sentence vector. This was passed into a 5-layer DNN head: 768 $\rightarrow$ 1024 $\rightarrow$ 512 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 12, with ReLU activations and Dropout(0.3) between each layer. A dual learning rate strategy was applied — a lower rate for the BERT layers to preserve pre-trained representations and a higher rate for the newly initialised DNN head. Table 4.5.2 summarises the training configuration.

Parameter	Value
Epochs	30
Batch Size	32 (effective: 128 with accumulation)
Gradient Accumulation Steps	4
Max Sequence Length	256
Optimizer	AdamW
LR (BERT layers)	2×10^{-5}
LR (DNN head)	1×10^{-3}
Weight Decay	1×10^{-4}
LR Scheduler	CosineAnnealingLR (T_max = 30)
Mixed Precision	fp16
Loss Function	CrossEntropyLoss
Dataset	Medium

Table 4.5.2: BERT + DNN Fine-tuning Hyperparameters

4.5.2 E5

4.5.2.1 With DNN (Frozen Embeddings)

The `intfloat/e5-base` model was used in fully frozen inference mode. Texts were prefixed with `passage:` before tokenization. Mean pooling over the last hidden state (masked by the attention mask) produced a 768-dimensional embedding per sample. These were passed into a DNN classifier head: $768 \to 512 \to 256 \to 128 \to 12$, trained with AdamW (lr = 10^{-3}) over 100 epochs with batch size 64 and CosineAnnealingLR scheduling.

4.5.2.2 With LoRA Fine-tuning

Rather than full fine-tuning, Low-Rank Adaptation (LoRA) was applied to the `intfloat/e5-base` transformer, inserting trainable low-rank adapter matrices into the query, key, and value projection layers of each attention block. This kept 99.6% of model parameters frozen, with only 442,368 trainable parameters out of 109.9 million total (0.40%). The LoRA-adapted encoder output was mean-pooled and passed through Dropout(0.2) and a single linear classification layer. Table 4.5.3 summarises the training configuration.

4.5.3 BiLSTM with Attention and BERT Embeddings

The pre-trained word embedding layer from frozen `bert-base-uncased` (768-dim, vocab size 30,522) was used as input to a 2-layer Bidirectional LSTM. Sequences were tokenized

Parameter	Value
Epochs	12 (early stopping, patience = 4)
Batch Size	48
Max Sequence Length	256
LoRA Rank (r)	8
LoRA Alpha	16
LoRA Target Modules	query, key, value
LoRA Dropout	0.1
Optimizer	AdamW
Learning Rate	2×10^{-4}
Weight Decay	1×10^{-4}
LR Scheduler	Cosine with 10% warmup
Mixed Precision	fp16
Loss Function	CrossEntropyLoss
Dataset	Medium

Table 4.5.3: E5 + LoRA Fine-tuning Hyperparameters

using BERT's WordPiece tokenizer with maximum length 384 tokens (chosen at the 90th percentile of training text lengths), padded to fixed length. A custom additive attention mechanism aggregated LSTM outputs into a 512-dimensional context vector, with padding positions masked using -10^9 before softmax. The classifier head was: Linear(512 $\rightarrow$ 256) $\rightarrow$ LayerNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.4) $\rightarrow$ Linear(256 $\rightarrow$ 12). Table 4.5.4 summarises the training configuration.

4.5.4 BiGRU with Learned Embeddings

This model replaced BERT embeddings with a randomly initialised, fully trainable `nn.Embedding` layer (vocab size = 30,522, dim = 256). The same BERT WordPiece tokenizer and 384-token maximum length were retained for consistency. The encoder was a 2-layer Bidirectional GRU (hidden dim = 256 per direction, output dim = 512) followed by the same additive attention mechanism and MLP classifier head as the BiLSTM model. Table 4.5.5 summarises the training configuration.

4.5.5 TF-IDF + Deep Neural Networks

TF-IDF features (max 50,000 features, n-gram range (1, 3)) were paired with neural networks of eight different hidden layer configurations, ranging from (32, 16) up to (4096, 2048) neurons. Each network followed the architecture: Input $\rightarrow$ Dense(L_1) $\rightarrow$ Batch-

Parameter	Value
Epochs	8
Batch Size	64
Max Sequence Length	384
LSTM Hidden Dim (per direction)	256
LSTM Layers	2
Optimizer	AdamW
Learning Rate	3×10^{-4}
Weight Decay	1×10^{-4}
LR Scheduler	CosineAnnealingLR (T_max = 8)
Gradient Clipping	max_norm = 1.0
Dropout	0.4
Loss Function	CrossEntropyLoss
Dataset	Full

Table 4.5.4: BiLSTM + Attention Hyperparameters

Norm → Dropout(0.5) → Dense(L_2) → BatchNorm → Dropout(0.3) → Dense(12, softmax). All models were compiled with the Adam optimiser and sparse categorical cross-entropy loss, batch size 32, and early stopping on validation loss. Sparse TF-IDF matrices were converted to dense batches on-the-fly via a custom data generator.

4.6 Training Environment

Track 1 and the BiGRU/BiLSTM experiments in Track 3 were run on Kaggle GPU instances (NVIDIA P100/T4). BERT-based and E5-based Track 3 experiments were run on Google Colab with a T4 GPU (16GB VRAM). All PyTorch models used `torch.cuda` with automatic device detection. Mixed precision training (fp16) with `GradScaler` was applied to BERT+DNN and E5+LoRA models. Model checkpointing was performed after every epoch based on best validation accuracy, enabling resume-capable training across interrupted sessions.

Parameter	**Value**
Epochs	8
Batch Size	64
Max Sequence Length	384
Embedding Dim	256 (learned from scratch)
GRU Hidden Dim (per direction)	256
GRU Layers	2
Optimizer	AdamW
Learning Rate	3×10^{-4}
Weight Decay	1×10^{-4}
LR Scheduler	CosineAnnealingLR (T_max = 8)
Gradient Clipping	max_norm = 1.0
Dropout	0.4
Loss Function	CrossEntropyLoss
Dataset	Medium

Table 4.5.5: BiGRU + Attention Hyperparameters

Chapter 5

Results

This chapter presents the classification performance of all models across the three experimental tracks. Results are reported in terms of training accuracy, validation accuracy, test accuracy, and macro-averaged F1 score where available. All models target 12-class authorship attribution. For models where complete result logging was unavailable due to computational constraints or interrupted runs, accuracy is reported as <50%.

5.1 Track 1: Classical and Hybrid Methods

Table 5.1.1 summarises the performance of classical and hybrid feature-based models trained and evaluated on the full dataset.

Model	**Train Acc**	**Val Acc**	**Test Acc**	**Val F1**
Neural Network (TF-IDF)	91.55%	81.19%	81.19%	0.8204
SVM (TF-IDF)	90.69%	77.80%	77.80%	0.7864
LightGBM (W2V + Stylo.)	89.94%	76.95%	76.95%	0.7685
MLP (W2V + Stylo.)	60.67%	60.26%	60.26%	0.5946

Table 5.1.1: Track 1 — Full Dataset Results

The Neural Network trained on TF-IDF features achieved the strongest overall performance in this track, reaching a test accuracy of 81.19% and a macro-F1 score of 0.8204. The LinearSVC, also relying on TF-IDF features, followed with a strong test accuracy of 77.80% and a macro-F1 of 0.7864, marginally outperforming the hybrid LightGBM model (76.95%). All of these models significantly outperformed the MLP (60.26%), which struggled to utilise the combined Word2Vec and stylometric feature space effectively.

Further detailing the top-performing model, the neural network achieved a macro-precision of 84.17% and a macro-recall of 81.19%. As outlined in Table 5.1.2, this optimal accuracy of 81.19% was reached using a deep configuration of 4096 and 2048 neurons in its hidden layers, demonstrating that neural networks can effectively leverage the high-dimensional

TF-IDF feature space when given sufficient capacity.

However, the high accuracy of these top-performing TF-IDF models—both the neural network and the SVM—should be interpreted with caution. TF-IDF is a frequency-based representation, and as the EDA in Chapter 1 demonstrates, the dataset exhibits uneven word and sentence length distributions across classes; GPT-3, for instance, averages just 131 words while LLaMA-Chat averages 255. The strong performance observed in Track 1 is therefore likely partially attributable to the models learning these length-correlated frequency biases rather than genuine semantic or stylistic distinctions between LLM sources. This makes frequency-based models less reliable as general-purpose authorship attributors, and motivates the exploration of deeper, semantics-aware representations in Tracks 2 and 3.

Hidden Layer Configuration (of Neural Net.)		**Test Acc**
Layer 1 - 512 neurons	Layer 2 - 256 neurons	79.47%
Layer 1 - 1024 neurons	Layer 2 - 512 neurons	80.30%
Layer 1 - 2048 neurons	Layer 2 - 1024 neurons	80.89%
Layer 1 - 4096 neurons	Layer 2 - 2048 neurons	81.19%

Table 5.1.2: Track 1 — TF-IDF with Neural Network Results

5.2 Track 2: Sentence Embeddings with Lightweight Classifiers

Results across all embedding-classifier combinations are detailed in Table 5.2.1, with test accuracies ranging from 31.07% to a peak of 63.01% (achieved by the E5 embedding paired with an MLP classifier). While all configurations perform well above the random baseline threshold for a 12-class problem (8.33%), the overall accuracy remains relatively moderate, with only a few classifiers (namely Random Forest and MLP) consistently exceeding the 50% mark. These experiments serve to establish a baseline for frozen sentence embeddings and motivate the shift toward fine-tuned transformer approaches in Track 3.

Embeddings →	MiniLM	MPNet	BGE	E5
Classifiers ↓				
Logistic Regression	31.68	35.41	41.01	47.00
SVM	35.75	31.75	41.45	48.38
Random Forest	55.35	56.42	56.98	57.72
MLP	46.72	52.13	58.43	63.01
RNN	31.28	35.65	40.14	45.77
LSTM	39.41	41.74	46.78	47.23
BiLSTM	41.03	43.93	49.25	49.40
GRU	38.97	41.23	46.56	47.12
Text CNN	31.07	31.19	40.33	46.30

Table 5.2.1: Track 2 — Sentence Embedding Results

5.3 Track 3: Transformer-Based Deep Learning Models

Table 5.3.1 summarises the performance of all transformer-based models. The BERT+GRU model was dropped due to prohibitive training time of multiple days — a single epoch did not complete within available compute constraints.

Model	Dataset	Best Val Acc	Test Acc	Epochs
BERT + GRU	Small/Medium	NA	Dropped	NA
BERT + DNN Finetune	Large	69.73%	69.56%	24
E5 + DNN (Frozen)	Medium	60.92%	60.92%	100
E5 + DNN (Fine Tuned)	Medium	66.36%	66.36%	15
BiLSTM + Attention	Full	81.44%	81.44%	8
BiGRU + Attention	Medium	87.13%	87.13%	8

Table 5.3.1: Track 3 — Transformer-Based Model Results

5.3.1 BERT + DNN Fine-tuning

Evaluated on the large test set (98,220 samples), the BERT+DNN fine-tuning model achieved an overall test accuracy of 69.56% with a macro-averaged F1 score of 0.6959. The per-class F1 scores are shown in Figure 5.2 and the normalised confusion matrix in Figure 5.1. The full precision, recall, and F1 breakdown per class is shown in Figure 5.3.

Several notable patterns emerge from the per-class results. GPT-4 achieved the highest F1 score (0.817), closely followed by LLaMA-Chat (0.769) and ChatGPT (0.763), suggesting

that these sources have sufficiently distinct linguistic signatures for the model to reliably separate them. In contrast, Cohere (0.570) and Cohere-Chat (0.579) were the most difficult classes, with the confusion matrix confirming substantial cross-confusion between the two — consistent with their shared underlying model family.

A particularly striking result is the human class: the model achieves very high precision (0.905) but low recall (0.583), meaning it is highly conservative about predicting a sample as human-authored and frequently misclassifies human text as LLM-generated, most often as Cohere. This asymmetry suggests the model has learned a narrow, high-confidence representation of human writing rather than a broad one. Mistral and MPT also show notable cross-confusion with GPT-2, likely due to overlapping sentence length distributions noted in the EDA.

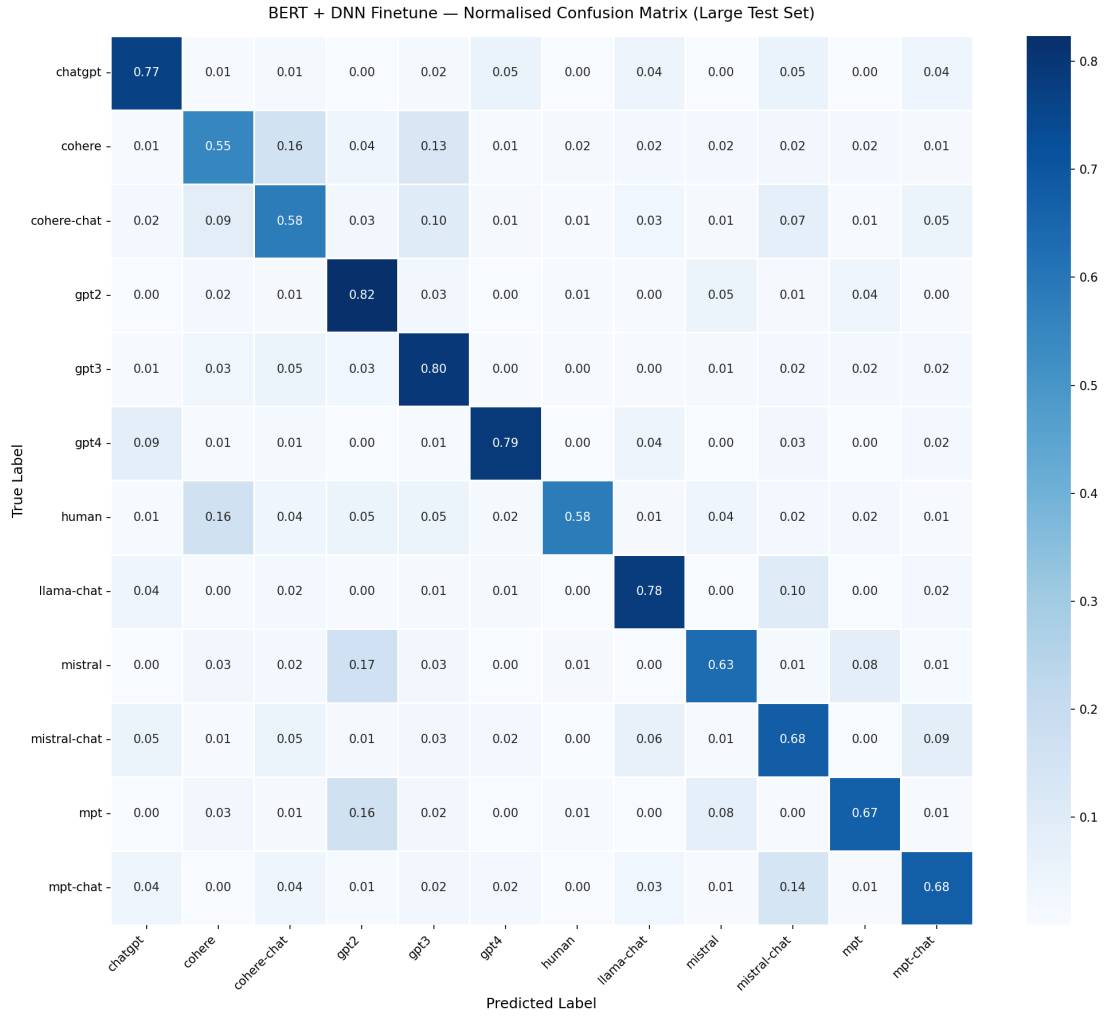


Figure 5.1: BERT + DNN Finetune — Row-normalised confusion matrix on the large test set. Diagonal values represent per-class recall.

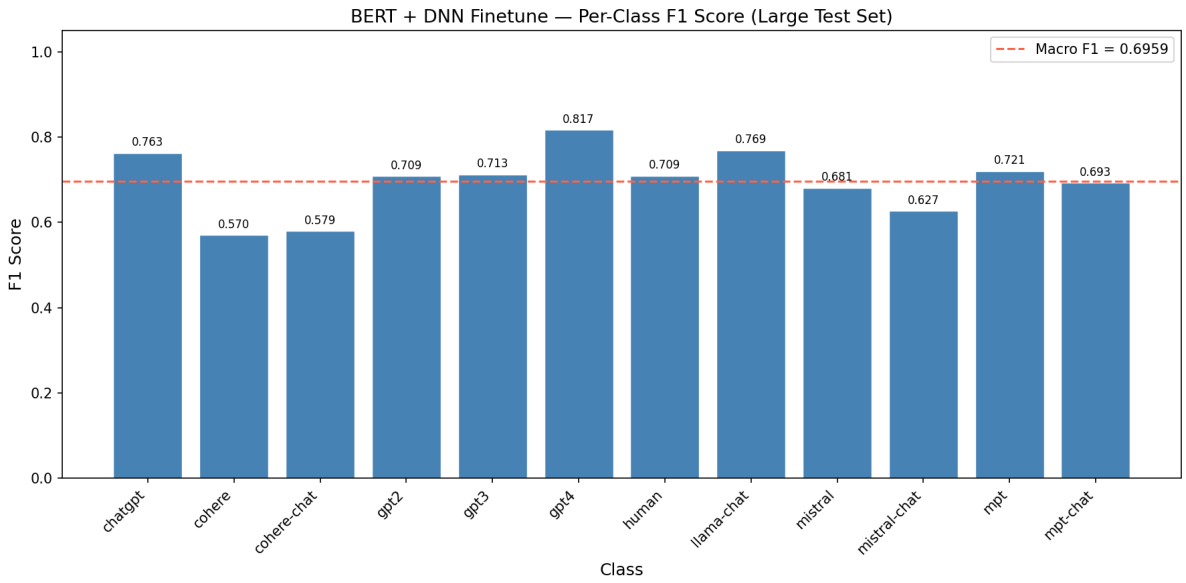


Figure 5.2: BERT + DNN Finetune — Per-class F1 scores on the large test set. The dashed line indicates the macro F1 of 0.6959.

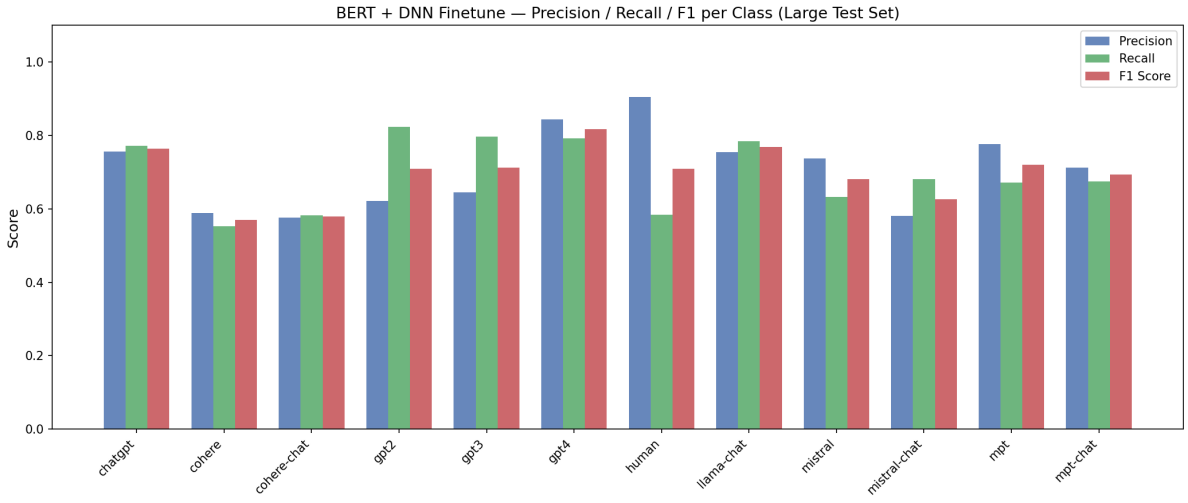


Figure 5.3: BERT + DNN Finetune — Precision, recall, and F1 score per class on the large test set.

5.3.2 BiLSTM with Attention and BiGRU with Attention

The BiGRU with additive attention and learned embeddings achieved the highest overall performance at 87.13% validation accuracy, followed by the BiLSTM with frozen BERT embeddings at 81.44%. Both models were trained for only 8 epochs and showed no signs of plateauing at convergence, suggesting further gains are likely with additional training. Notably, the BiGRU outperformed the BiLSTM despite using randomly initialised embeddings rather than pre-trained BERT representations, indicating that the GRU architecture and the larger training data contributed meaningfully to performance.

5.3.3 Other Models

The frozen E5+DNN baseline reached 60.92% test accuracy, confirming that even high-quality sentence embeddings benefit significantly from task-specific fine-tuning. The TF-IDF+NN configurations across all 8 layer sizes failed to produce competitive results, reinforcing the finding from Track 1 that sparse lexical features are insufficient without either powerful classifiers or substantially larger feature spaces than those explored here.

5.4 Overall Comparison

Track	**Best Model**	**Test Acc**
Track 1 — Classical	Neural Network (TF-IDF)	81.19%
Track 2 — Sent. Embed.	E5 (MLP)	63.01%
Track 3 — Transformer	BiGRU + Attention	87.13%

Table 5.4.1: Best Model Per Track

Across all tracks, the BiGRU with additive attention achieved the best overall test accuracy of 87.13%, establishing it as the strongest model in this study. The classical Neural Network baseline using TF-IDF features (81.19%) remained highly competitive against the transformer-based approaches, though as discussed above, its accuracy is likely inflated by length-distribution bias in the TF-IDF feature space rather than genuine authorship discrimination. The newly incorporated sentence embedding approaches (Track 2) peaked at 63.01% using the E5 embedding with an MLP classifier, serving as a baseline for dense representations but falling short of the top-performing architectures. Meanwhile, the BERT+DNN fine-tuning model (69.56%), while lower in raw accuracy than the leading models, is considered a more semantically meaningful result given its reliance on contextual representations rather than surface frequency statistics, and its performance was still improving at epoch 24 when training was halted.

Chapter 6

Conclusion

This work presented a systematic investigation into the problem of LLM authorship attribution — classifying a given text sample as having been generated by one of eleven large language models or authored by a human. We explored three broad experimental tracks spanning classical feature-based methods, sentence embedding approaches, and transformer-based deep learning models, evaluated on a large-scale balanced dataset of 996,000 samples across 12 classes.

6.1 Summary of Findings

The experiments revealed a clear performance hierarchy across the three tracks. Within the classical and hybrid methods, a deep Neural Network trained on TF-IDF features achieved the strongest performance, reaching a test accuracy of 81.19% and a macro-F1 score of 0.8204. However, the strong performance of this TF-IDF based model is partially attributable to length-distribution bias in the feature space — the model exploits uneven word count distributions across classes rather than genuine stylistic or semantic differences between sources. This limits the practical reliability of such approaches as general-purpose authorship attributors.

Among the transformer-based approaches in Track 3, the BiGRU with additive attention and learned embeddings achieved the highest overall accuracy of 87.13%, demonstrating that a relatively lightweight recurrent architecture trained with sufficient data can outperform more parameter-heavy transformer fine-tuning pipelines, particularly under constrained compute budgets. The BiLSTM with frozen BERT embeddings followed at 81.44%, confirming the value of pre-trained semantic representations even when used without fine-tuning the full transformer stack.

The BERT+DNN fine-tuning model, evaluated on the large test set, achieved 69.56% accuracy with a macro F1 of 0.6959. Per-class analysis revealed that semantically distinct sources such as GPT-4 (F1: 0.817) and LLaMA-Chat (F1: 0.769) are well-separated,

while model family pairs such as Cohere and Cohere-Chat (F1: 0.570, 0.579) remain difficult to distinguish — a finding consistent with their shared underlying architecture and training data. The human class exhibited high precision but low recall, suggesting the model has learned an overly conservative representation of human writing that misclassifies a significant fraction of human-authored text as LLM-generated.

6.2 Limitations

Several limitations constrain the conclusions of this study. First, the majority of transformer-based models were trained on the medium dataset subset rather than the full dataset due to GPU compute constraints on Colab and Kaggle free tiers, meaning their reported accuracies are not directly comparable to the classical models trained on the full corpus. Second, training for models such as BERT+DNN was terminated before convergence — the model was still improving at epoch 24 — so reported accuracies represent lower bounds on achievable performance. Third, the Track 2 sentence embedding experiments were lost to session interruptions and could not be completed, leaving a gap in the experimental comparison. Finally, the dataset itself may contain prompt-distribution artifacts that make certain classes easier to identify based on topic or format rather than generative style, which would inflate accuracy estimates across all models.

6.3 Future Work

Several directions present clear opportunities for improvement. Continued training of the BERT+DNN and E5+LoRA models beyond their current checkpoints is the most immediate next step, as both were still improving at termination. Extending the BiGRU and BiLSTM models to the full dataset would provide a fairer comparison against the classical baselines. More aggressive regularisation — higher dropout, label smoothing, or data augmentation — could address the growing train-validation gap observed in the BiGRU from epoch 6 onwards. At the architecture level, replacing the additive attention mechanism with multi-head self-attention or experimenting with full transformer fine-tuning using more efficient methods such as QLoRA could yield further gains. Finally, a more thorough analysis of per-class confusion patterns — particularly the Cohere/Cohere-Chat and Mistral/MPT pairs — could motivate hierarchical classification strategies that first identify the model family before attributing the specific variant.

Appendix A

Dataset Analysis Figures

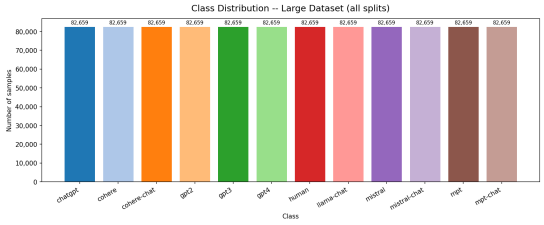


Figure A.1: Class distribution across all splits of the large dataset. Each class contains exactly 82,659 samples.

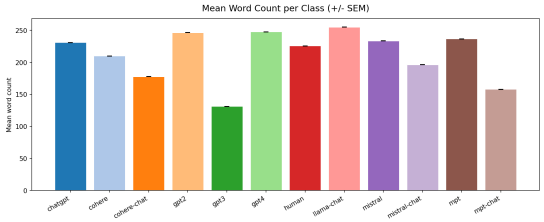


Figure A.2: Mean word count per class with standard error of the mean (SEM) error bars.

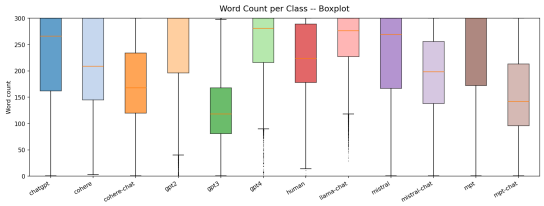


Figure A.3: Box plot of word count distributions per class, showing median, interquartile range, and outliers.

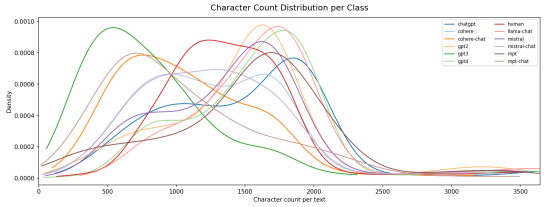


Figure A.4: Character count distribution per class shown as kernel density estimates.

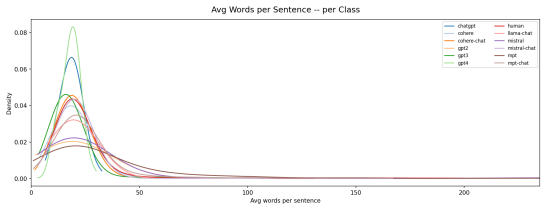


Figure A.5: Distribution of average words per sentence across classes.

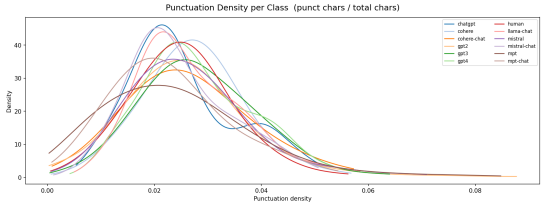


Figure A.6: Punctuation density distributions per class (punctuation characters / total characters).